Pontificia Universidad Javeriana

Polinomios de Lagrange y trazadores cúbicos: interpolación de la población de Croacia (1994–2023)

Análisis Numérico — Proyecto Final
Salomón Alfredo Ávila Larrotta
Noviembre de 2026

Resumen. Se construyen, a partir de 25 datos anuales de población de Croacia, un polinomio interpolante de Lagrange de grado 24 y un trazador cúbico con condiciones de frontera naturales, y se usan ambos para estimar la población en cinco años reservados (1995, 2000, 2008, 2015 y 2022). En el interior del intervalo los dos métodos alcanzan errores relativos del orden de 10^{-4}; cerca de los extremos, en cambio, el polinomio de Lagrange oscila con violencia (fenómeno de Runge) y produce errores relativos de 12,3 y 8,3, es decir, de más del 800 %. El trazador cúbico mantiene errores por debajo de $1{,}1 \times 10^{-3}$ en los cinco años.

1. Introducción

Interpolar consiste en construir una función que pase exactamente por un conjunto de datos conocidos, para luego usarla en la estimación de valores intermedios. Cuando los datos son numerosos surgen dos estrategias naturales: usar un único polinomio que pase por todos los puntos (polinomio de Lagrange) o unir muchos polinomios de grado bajo, uno por subintervalo, de forma suave (trazador cúbico). El objetivo de este trabajo es comparar ambas estrategias sobre un caso real: la población de Croacia entre 1994 y 2023.

El archivo de datos del curso contiene 30 parejas (año, población). Cinco de ellas están resaltadas: 1995, 2000, 2008, 2015 y 2022. Estos cinco datos se excluyen de la construcción de los interpolantes y se reservan para medir la calidad de las aproximaciones mediante el error relativo

$$E_r = \frac{|p - p^*|}{|p|}, \tag{1}$$

donde p es el valor exacto y p^* el aproximado.

2. Implementación

2.1. Elección del lenguaje y organización del programa

Los algoritmos se implementaron en **C++** (estándar C++17). Se eligió este lenguaje por su velocidad, por el control explícito que ofrece sobre los tipos numéricos (en particular, la precisión extendida `long double`) y porque basta un compilador estándar, como `g++`, `clang++` o el de Visual Studio, para ejecutar los programas en cualquier sistema operativo. Los dos métodos se programaron desde cero, usando solo la librería estándar (`<vector>`, `<cmath>` y `<cstdio>`), sin rutinas de interpolación externas.

Cada algoritmo está en un único archivo autocontenido, que incluye los datos y todas sus dependencias:

Archivo	Función
`lagrange.cpp`	Construcción y evaluación del polinomio de Lagrange; imprime la tabla de errores y exporta los resultados y la curva en formato CSV.
`trazador_cubico.cpp`	Cálculo de los coeficientes del trazador natural, evaluación, tabla de errores y exportación a CSV.
`graficas.py`	Script auxiliar que lee los CSV producidos por los programas en C++ y dibuja las figuras de este informe.

Cada programa se compila y se ejecuta de inmediato:

```
g++ -std=c++17 -O2 -o lagrange lagrange.cpp && ./lagrange
g++ -std=c++17 -O2 -o trazador_cubico trazador_cubico.cpp && ./trazador_cubico
```

Ambos programas se organizan de la misma forma: (i) un bloque de datos, (ii) la función `separarDatos`, que divide los datos en nodos y datos de prueba, (iii) las funciones del método numérico y (iv) la función `main`, que calcula las aproximaciones, el error relativo y escribe los archivos de salida. El código completo se incluye en el Anexo.

2.2. Captura de los datos

Los 30 valores de población de la hoja `CROACIA` del archivo Excel del curso se copiaron en el arreglo constante `POBLACION`, indexado desde 1994, y los años resaltados se guardaron en el arreglo `SUBRAYADOS = {1995, 2000, 2008, 2015, 2022}`. Escribir los datos dentro del código evita depender de librerías para leer archivos de Excel, que no forman parte de la librería estándar de C++, y garantiza que el programa corra sin archivos adicionales.

La función `separarDatos` recorre los 30 años y, con ayuda de `esSubrayado(anio)`, los reparte en dos grupos de vectores:

- `xNodos`, `yNodos`: los 25 años no resaltados y su población (datos de interpolación);
- `anioPrueba`, `yPrueba`: los 5 años resaltados y su población exacta.

2.3. Polinomio de Lagrange

Dados $n+1$ nodos distintos $(x_0, y_0), ..., (x_n, y_n)$, el polinomio de Lagrange es el único polinomio de grado a lo sumo n que pasa por todos ellos:

$$P(x) = \sum_{k=0}^{n} y_k L_k(x), \qquad L_k(x) = \prod_{j=0, j\neq k}^{n} \frac{x - x_j}{x_k - x_j}. \tag{2}$$

Cada polinomio base L_k vale 1 en x_k y 0 en los demás nodos, por lo que $P(x_k) = y_k$. Con 25 nodos se obtiene un polinomio de grado 24.

La implementación sigue literalmente la ecuación 2: la función `baseLagrange(k, x, nodos)` acumula el producto que define $L_k(x)$, y la función `lagrange(x, nodos, valores)` realiza la suma ponderada. Se tomaron dos decisiones para que el resultado sea confiable:

1. **Traslación de la variable.** En lugar de trabajar con años como 2023, se usa $t = \text{año} - 1994$, de modo que $t \in [0, 29]$. Esto no cambia el polinomio, pero evita operar con números grandes.
2. **Precisión extendida.** Todos los cálculos se hacen con el tipo `long double` (alias `real` en el código), que en `g++` ofrece unas 19 cifras significativas en lugar de las 16 de `double`. Al imprimir, los valores se convierten a `double`, porque algunos compiladores de Windows (MinGW) no muestran correctamente `long double` con `printf`.

```
// Evalúa el k-ésimo polinomio base L_k en el punto x.
real baseLagrange(int k, real x, const std::vector<real>& nodos) {
    real L = 1.0L;
    for (size_t j = 0; j < nodos.size(); j++) {
        if ((int)j != k) L *= (x - nodos[j]) / (nodos[k] - nodos[j]);
    }
    return L;
}

// Evalúa el polinomio interpolante P(x) = sum_k y_k L_k(x).
real lagrange(real x, const std::vector<real>& nodos, const std::vector<real>& valores) {
    real suma = 0.0L;
    for (size_t k = 0; k < nodos.size(); k++)
```

```
        suma += valores[k] * baseLagrange((int)k, x, nodos);
    return suma;
}
```

Evaluar P en un punto cuesta $O(n^2)$ operaciones, lo cual es despreciable para 25 nodos.

2.4. Trazador cúbico natural

Un trazador cúbico S es una función definida a trozos: en cada subintervalo $[x_j, x_{j+1}]$, con $j = 0, \ldots, n-1$,

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3. \tag{3}$$

Se exige que S interpole los datos y que S, S' y S'' sean continuas en los nodos interiores. Las condiciones de frontera naturales son $S''(x_0) = S''(x_n) = 0$, lo que equivale a $c_0 = c_n = 0$.

Con $h_j = x_{j+1} - x_j$ y $a_j = y_j$, estas condiciones se reducen a un sistema lineal tridiagonal en las incógnitas c_j:

$$h_{j-1}c_{j-1} + 2(h_{j-1} + h_j)c_j + h_j c_{j+1} = \frac{3}{h_j}(a_{j+1} - a_j) - \frac{3}{h_{j-1}}(a_j - a_{j-1}), \tag{4}$$

para $j = 1, \ldots, n-1$. La matriz es estrictamente diagonal dominante, así que el sistema tiene solución única. Una vez conocidos los c_j,

$$b_j = \frac{a_{j+1} - a_j}{h_j} - \frac{h_j}{3}(c_{j+1} + 2c_j), \qquad d_j = \frac{c_{j+1} - c_j}{3h_j}. \tag{5}$$

Los coeficientes se agrupan en la estructura `Trazador`, que guarda los nodos y los vectores `a`, `b`, `c` y `d`. La función `trazadorNatural(x, y)` implementa el Algoritmo 3.4 de Burden y Faires. El sistema tridiagonal se resuelve con la factorización de Crout (algoritmo de Thomas) en tres fases: cálculo del lado derecho `alfa`, eliminación hacia adelante (`l`, `mu`, `z`) y sustitución hacia atrás para obtener c, b y d:

```
// Pasos 3-5: factorización de Crout y sustitución hacia adelante.
for (int i = 1; i < n; i++) {
    l[i] = 2.0 * (x[i + 1] - x[i - 1]) - h[i - 1] * mu[i - 1];
    mu[i] = h[i] / l[i];
    z[i] = (alfa[i] - h[i - 1] * z[i - 1]) / l[i];
}
// Paso 6: sustitución hacia atrás para c, y cálculo de b y d.
for (int j = n - 1; j >= 0; j--) {
    c[j] = z[j] - mu[j] * c[j + 1];
    b[j] = (a[j + 1] - a[j]) / h[j] - h[j] * (c[j + 1] + 2.0 * c[j]) / 3.0;
    d[j] = (c[j + 1] - c[j]) / (3.0 * h[j]);
}
```

El costo es $O(n)$. Por su parte, `evaluarTrazador(t, S)` localiza el subintervalo que contiene a t y evalúa el polinomio cúbico correspondiente. Como los nodos no son equiespaciados (faltan los años reservados), las longitudes h_j valen 1 o 2 años; el algoritmo admite este caso sin modificaciones.

2.5. Medición del tiempo de ejecución

Cada programa mide, con el reloj monótono `std::chrono::steady_clock` de la librería estándar, cuánto tarda cada etapa del algoritmo. Solo se cronometran los cálculos: la lectura de los datos, la impresión en consola y la escritura de archivos quedan por fuera. Las etapas medidas son:

- **Construcción**: cálculo de los coeficientes a_j, b_j, c_j, d_j del trazador con `trazadorNatural`. El polinomio de Lagrange, en la forma de la ecuación 2, no tiene esta etapa: no precalcula nada y hace todo el trabajo en cada evaluación.

- **Evaluación en los 5 años de prueba**: las cinco aproximaciones de la Tabla 1.
- **Evaluación en la malla**: los 2001 puntos con los que se dibujan las curvas de las figuras.

Una sola ejecución de estas etapas puede durar menos de un microsegundo, por debajo de la resolución práctica del reloj. Por eso la función `medirMicrosegundos` repite cada etapa muchas veces (entre 200 y 100 000) y reporta el tiempo promedio por repetición. Para que el compilador, con la optimización `-O2`, no elimine cálculos cuyo resultado no se usa, cada resultado se guarda en una variable `volatile`:

```
template <typename F>
double medirMicrosegundos(F f, int reps) {
    auto inicio = std::chrono::steady_clock::now();
    for (int r = 0; r < reps; r++) f();
    auto fin = std::chrono::steady_clock::now();
    return std::chrono::duration<double, std::micro>(fin - inicio).count() / reps;
}
```

Los tiempos se imprimen en consola y se exportan a `tiempos_lagrange.csv` y `tiempos_trazador.csv`.

2.6. Verificación

Para comprobar las implementaciones, sus resultados se compararon con cálculos independientes: el polinomio de Lagrange se evaluó también en aritmética racional exacta (sin redondeo alguno) y con la rutina `BarycentricInterpolator` de SciPy, y el trazador con `CubicSpline` de SciPy usando `bc_type="natural"`. Los valores coinciden en todas las cifras significativas reportadas. En particular, la coincidencia con la aritmética exacta muestra que el redondeo no influye en los resultados del polinomio de Lagrange.

3. Resultados

La Tabla 1 resume las aproximaciones obtenidas en los cinco años reservados.

		Lagrange		Trazador cúbico	
Año	**Exacto**	**Aproximado**	**Error rel.**	**Aproximado**	**Error rel.**
1995	4 619 362	61 303 888	$1{,}227 \times 10^{1}$	4 619 106	$5{,}542 \times 10^{-5}$
2000	4 326 105	4 327 519	$3{,}269 \times 10^{-4}$	4 330 669	$1{,}055 \times 10^{-3}$
2008	4 314 635	4 313 818	$1{,}893 \times 10^{-4}$	4 314 507	$2{,}976 \times 10^{-5}$
2015	4 183 732	4 184 598	$2{,}070 \times 10^{-4}$	4 184 196	$1{,}108 \times 10^{-4}$
2022	3 907 031	36 302 718	$8{,}292 \times 10^{0}$	3 906 545	$1{,}244 \times 10^{-4}$

Tabla 1: Población exacta y aproximada de Croacia (habitantes) en los años reservados, con el error relativo de cada método.

La Figura 1 muestra ambos interpolantes en todo el intervalo junto con los 30 datos exactos. La escala vertical está dominada por las oscilaciones del polinomio de Lagrange, que superan los 150 millones de habitantes cerca de 1994 cuando la población real es de unos 4,6 millones.

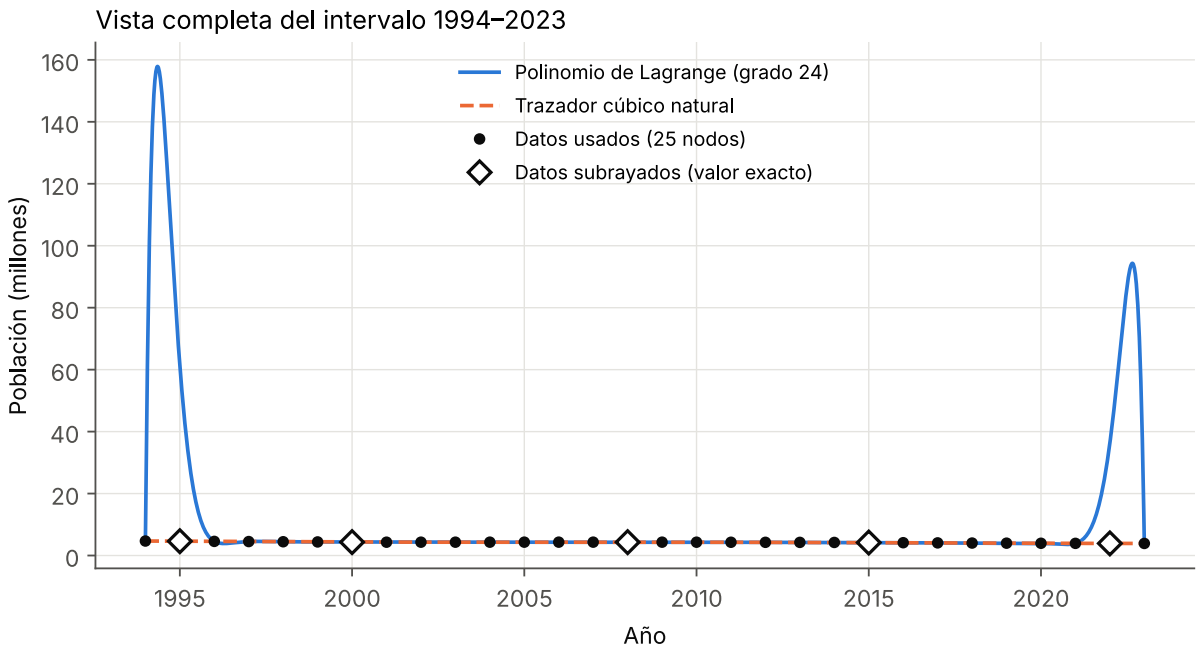


Figura 1: Vista completa: el polinomio de Lagrange de grado 24 oscila con fuerza en el primer y el último subintervalo.

La Figura 2 repite la gráfica en la escala de los datos. El trazador cúbico (línea discontinua) sigue la tendencia en todo el periodo, mientras que el polinomio de Lagrange solo se comporta bien entre aproximadamente 1999 y 2017.

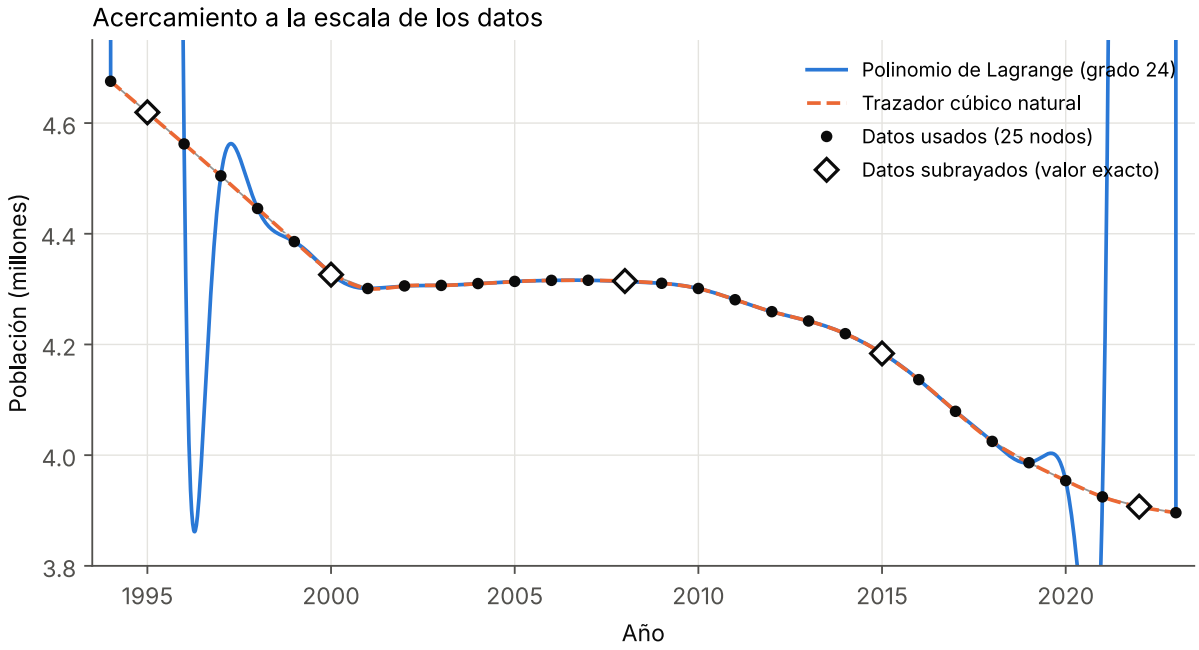


Figura 2: Acercamiento a la escala de los datos. Los rombos son los valores exactos de los años reservados.

Finalmente, la Figura 3 compara los errores relativos en escala logarítmica.

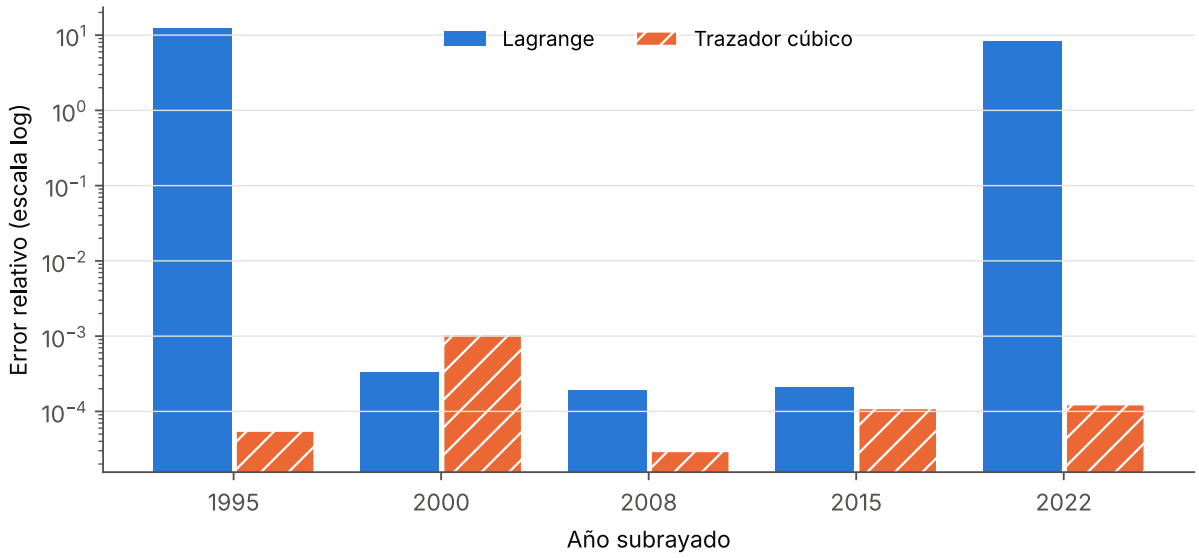


Figura 3: Error relativo de cada método en los años reservados (escala logarítmica).

3.1. Tiempo de ejecución

Los programas se compilaron y ejecutaron en el equipo descrito en la Tabla 2. Ambos programas son secuenciales, así que usan un solo hilo del procesador. Las mediciones se hicieron con el portátil conectado a la corriente y con el perfil de energía en modo de alto rendimiento.

Equipo	Portátil ASUS TUF Dash F15 (FX517ZC)
Procesador	Intel Core i5-12450H (12.ª generación): 8 núcleos (4 de rendimiento y 4 de eficiencia), 12 hilos, hasta 4,4 GHz
Memoria caché	L1 de datos 320 KiB, L2 7 MiB, L3 12 MiB
Memoria RAM	32 GB
Sistema operativo	Fedora Linux 44 (núcleo Linux 7.2), 64 bits
Compilador	`g++` (GCC) 16.2.1, opciones `-std=c++17 -O2`

Tabla 2: Especificaciones del equipo en el que se ejecutaron los programas.

La Tabla 3 reúne los tiempos medidos. Los valores cambian de una máquina a otra y, en menor medida, de una ejecución a otra (en corridas repetidas variaron menos de un 10 %), pero las proporciones entre los dos métodos se mantienen.

Etapa	**Lagrange** (µs)	**Trazador cúbico** (µs)
Construcción de coeficientes	—	0,340
Evaluación en los 5 años de prueba	4,317	0,038
Evaluación en la malla de 2001 puntos	1681,6	18,5
Tiempo por evaluación (malla)	840 ns	9,2 ns

Tabla 3: Tiempo promedio de ejecución de cada etapa, en microsegundos (1 µs = 10^{-6} s). La última fila divide el tiempo de la malla entre sus 2001 puntos.

Los dos métodos son prácticamente instantáneos para este problema: incluso la etapa más lenta, evaluar el polinomio de Lagrange en los 2001 puntos de la malla, tarda menos de 2 milisegundos. Aun así, la diferencia relativa es grande. Calcular los coeficientes del trazador cuesta apenas 0,34 µs, y después cada evaluación del trazador es unas 91 veces más rápida que una del polinomio de Lagrange. Incluso sumando la construcción, el trazador obtiene las cinco estimaciones de la Tabla 1 unas 11 veces más rápido.

4. Análisis y comparación

4.1. Diferencias observadas

En los extremos (1995 y 2022) la diferencia es enorme. El polinomio de Lagrange estima 61,3 millones de habitantes en 1995 y 36,3 millones en 2022, frente a valores reales de 4,6 y 3,9 millones: errores relativos de 12,3 y 8,3. El trazador cúbico, en cambio, comete errores de $5{,}5 \times 10^{-5}$ y $1{,}2 \times 10^{-4}$, es decir, se equivoca en unos pocos cientos de habitantes.

En el interior (2000, 2008 y 2015) ambos métodos son buenos, con errores entre 10^{-5} y 10^{-3}. El trazador es mejor en 2008 y 2015. En 2000, sin embargo, el polinomio de Lagrange ($3{,}3 \times 10^{-4}$) supera al trazador ($1{,}1 \times 10^{-3}$), que es el único año en el que Lagrange gana.

4.2. ¿Por qué se producen estas diferencias?

El fenómeno de Runge. El error de la interpolación polinomial en un punto x es

$$f(x) - P(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i), \tag{6}$$

para algún ξ en el intervalo. El factor $\omega(x) = \prod(x - x_i)$ depende solo de la ubicación de los nodos, y para nodos (casi) equiespaciados crece de forma explosiva hacia los extremos del intervalo. Con los nodos de este problema, $|\omega|$ vale $8{,}7 \times 10^{24}$ en 1995 y $5{,}2 \times 10^{24}$ en 2022, pero apenas $1{,}1 \times 10^{19}$ en 2008: hay una diferencia de cinco a seis órdenes de magnitud entre el centro y los bordes. Para que un único polinomio de grado 24 pase por los 25 nodos, se ve obligado a oscilar con gran amplitud entre los nodos extremos, justo donde caen 1995 (entre 1994 y 1996) y 2022 (entre 2021 y 2023). Que los resultados en precisión extendida coincidan con los de la aritmética exacta confirma que se trata de una limitación del método y no de errores de redondeo.

La ventaja de trabajar por tramos. El trazador cúbico nunca usa polinomios de grado mayor que 3, y cada tramo depende sobre todo de los datos cercanos. Por eso no hay oscilaciones que se propaguen por el intervalo: un dato de 2023 tiene muy poca influencia sobre la estimación en 1995. La condición de frontera natural ($S'' = 0$ en los extremos) introduce un pequeño error cerca de los bordes, pero en este caso resultó insignificante.

El caso del año 2000. La población de Croacia baja casi linealmente, unos 60 000 habitantes por año, hasta el año 2000, y a partir de 2001 se estabiliza de forma abrupta en torno a 4,3 millones. Con el año 2000 excluido, este cambio de tendencia ocurre dentro del subintervalo $[1999, 2001]$, de dos años. El trazador, que está obligado a tener segunda derivada continua, suaviza ese quiebre y sobrestima la población en unos 4 500 habitantes. El polinomio de Lagrange también es suave, pero en esa zona central su comportamiento resultó estar más cerca del dato real. La ventaja es circunstancial y no contradice la conclusión general: ningún interpolante suave puede reproducir con exactitud un cambio brusco que ocurre entre dos nodos.

4.3. Comparación de costos

Los tiempos de la Tabla 3 coinciden con lo que predice el conteo de operaciones. El trazador resuelve un sistema tridiagonal una sola vez, con $O(n)$ operaciones, y luego cada evaluación solo localiza el subintervalo (a lo sumo n comparaciones) y evalúa un polinomio cúbico. El polinomio de Lagrange, en cambio, no reutiliza nada entre evaluaciones: cada una calcula los $n + 1$ polinomios base, cada uno con n productos y divisiones, es decir, $O(n^2)$ operaciones. Con $n = 24$ son unas $25 \times 24 = 600$ divisiones por punto, en precisión extendida (`long double`), que es más lenta que `double`. Eso explica que una evaluación de Lagrange tarde cerca de 840 ns frente a unos 9 ns del trazador.

La diferencia crecería con más datos: duplicar el número de nodos multiplica por cerca de cuatro el costo de cada evaluación de Lagrange, pero solo por dos el del trazador. El costo de Lagrange podría bajar a $O(n)$ por evaluación usando la fórmula baricéntrica, que precalcula unos pesos, pero eso no

corregiría su principal defecto: la estabilidad. Agregar datos mejora el trazador, mientras que en el polinomio de Lagrange con nodos equiespaciados suele empeorar las oscilaciones.

5. Conclusiones

1. Con muchos datos equiespaciados, el polinomio interpolante de Lagrange no es una herramienta adecuada para aproximar valores cerca de los extremos: en 1995 y 2022 produjo estimaciones entre 9 y 13 veces mayores que la población real.
2. El trazador cúbico natural aproximó los cinco años con errores relativos inferiores a $1{,}1 \times 10^{-3}$, y en cuatro de ellos inferiores a $1{,}2 \times 10^{-4}$.
3. El trazador también es más rápido: una vez calculados sus coeficientes, en menos de un microsegundo, cada evaluación es unas 91 veces más rápida que la del polinomio de Lagrange. Para 25 datos ambos tiempos son despreciables, pero la diferencia crece con el número de nodos.
4. En el centro del intervalo ambos métodos tienen una precisión comparable. Lagrange solo superó al trazador en el año 2000, debido a un cambio brusco de tendencia en los datos que ningún interpolante suave puede capturar por completo.
5. Para interpolar conjuntos numerosos de datos, como series de población, es preferible usar trazadores cúbicos u otros métodos por tramos en lugar de un único polinomio de grado alto.

Referencia. R. L. Burden, J. D. Faires y A. M. Burden, *Análisis Numérico*, 10.ª ed., Cengage Learning, 2017. Capítulo 3: Interpolación y aproximación polinomial.

Anexo: código fuente completo

`lagrange.cpp`

```cpp
/*
 * lagrange.cpp
 * ------------
 * Interpolación de la población de Croacia (1994-2023) con el polinomio de
 * Lagrange construido a partir de los 25 datos NO subrayados.
 *
 *     P(x) = sum_{k=0}^{n} y_k * L_k(x),
 *     L_k(x) = prod_{j != k} (x - x_j) / (x_k - x_j).
 *
 * Decisiones de implementación:
 *   - Los datos se escriben directamente en el código (tomados de la hoja
 *     CROACIA del Excel del curso), así el programa no depende de archivos
 *     externos ni de librerías adicionales.
 *   - La variable independiente se traslada a t = año - 1994 para no operar
 *     con potencias de números del orden de 2000.
 *   - Se usa long double (precisión extendida) para reducir el redondeo.
 *     Al imprimir se convierte a double, porque algunos compiladores de
 *     Windows (MinGW) no imprimen bien long double con printf.
 *
 * Compilar:  g++ -std=c++17 -O2 -o lagrange lagrange.cpp
 * Ejecutar:  ./lagrange        (en Windows: lagrange.exe)
 *
 * Salidas: tabla en consola, resultados_lagrange.csv, curva_lagrange.csv
 * (esta última sirve para graficar el polinomio en todo el intervalo) y
 * tiempos_lagrange.csv (tiempo de ejecución del algoritmo).
 */

#include <chrono>
#include <cmath>
#include <cstdio>
#include <vector>

// ---------------------------------------------------------------------------
// Datos: población de Croacia 1994-2023 (hoja CROACIA del archivo Excel).
// ---------------------------------------------------------------------------
const int N_DATOS = 30;
const int ANIO_INICIAL = 1994;
const long POBLACION[N_DATOS] = {
    4675517, 4619362, 4562290, 4504627, 4445669, 4385823, 4326105, 4301079,
    4305783, 4306683, 4310050, 4313946, 4315802, 4315959, 4314635, 4310537,
    4301150, 4280827, 4259233, 4242403, 4219485, 4183732, 4136438, 4079198,
    4024868, 3986338, 3953956, 3924607, 3907031, 3896023};
// Años subrayados (resaltados en el Excel): se reservan para medir el error.
const int SUBRAYADOS[5] = {1995, 2000, 2008, 2015, 2022};

typedef long double real;

bool esSubrayado(int anio) {
    for (int s : SUBRAYADOS)
        if (s == anio) return true;
    return false;
}

// Separa los 30 datos en nodos de interpolación (25) y datos de prueba (5).
void separarDatos(std::vector<real>& xNodos, std::vector<real>& yNodos,
                  std::vector<int>& anioPrueba, std::vector<real>& yPrueba) {
    for (int i = 0; i < N_DATOS; i++) {
        int anio = ANIO_INICIAL + i;
        if (esSubrayado(anio)) {
```

```
            anioPrueba.push_back(anio);
            yPrueba.push_back(POBLACION[i]);
        } else {
            xNodos.push_back(anio - ANIO_INICIAL);  // variable trasladada t
            yNodos.push_back(POBLACION[i]);
        }
    }
}

// ---------------------------------------------------------------------------
// Algoritmo de Lagrange
// ---------------------------------------------------------------------------

// Evalúa el k-ésimo polinomio base L_k en el punto x.
real baseLagrange(int k, real x, const std::vector<real>& nodos) {
    real L = 1.0L;
    for (size_t j = 0; j < nodos.size(); j++) {
        if ((int)j != k) L *= (x - nodos[j]) / (nodos[k] - nodos[j]);
    }
    return L;
}

// Evalúa el polinomio interpolante P(x) = sum_k y_k L_k(x).
real lagrange(real x, const std::vector<real>& nodos, const std::vector<real>& valores) {
    real suma = 0.0L;
    for (size_t k = 0; k < nodos.size(); k++) suma += valores[k] * baseLagrange((int)k, x,
nodos);
    return suma;
}

real errorRelativo(real exacto, real aprox) { return std::fabs(exacto - aprox) /
std::fabs(exacto); }

// ---------------------------------------------------------------------------
// Medición del tiempo de ejecución
// ---------------------------------------------------------------------------

// Las variables volatile impiden que el compilador elimine o saque del ciclo
// de repeticiones los cálculos cuyo resultado no se usa.
volatile real sumidero;
volatile real cero = 0.0L;

// Tiempo promedio, en microsegundos, de una ejecución de f. Se repite 'reps'
// veces porque una sola ejecución dura menos que la resolución del reloj.
template <typename F>
double medirMicrosegundos(F f, int reps) {
    auto inicio = std::chrono::steady_clock::now();
    for (int r = 0; r < reps; r++) f();
    auto fin = std::chrono::steady_clock::now();
    return std::chrono::duration<double, std::micro>(fin - inicio).count() / reps;
}

int main() {
    std::vector<real> xNodos, yNodos, yPrueba;
    std::vector<int> anioPrueba;
    separarDatos(xNodos, yNodos, anioPrueba, yPrueba);

    std::printf("Polinomio de Lagrange de grado %zu - Croacia\n", xNodos.size() - 1);
    std::printf("Nodos usados: %zu datos\n\n", xNodos.size());
    std::printf("%6s %14s %18s %16s\n", "Anio", "Exacto", "Lagrange", "Error relativo");

    FILE* res = std::fopen("resultados_lagrange.csv", "w");
```

```
    if (res) std::fprintf(res, "anio,exacto,aproximado,error_relativo\n");

    for (size_t i = 0; i < anioPrueba.size(); i++) {
        real t = anioPrueba[i] - ANIO_INICIAL;
        real aprox = lagrange(t, xNodos, yNodos);
        real err = errorRelativo(yPrueba[i], aprox);
        std::printf("%6d %14.0f %18.2f %16.6e\n", anioPrueba[i], (double)yPrueba[i],
(double)aprox, (double)err);
        if (res) std::fprintf(res, "%d,%.0f,%.2f,%.6e\n", anioPrueba[i], (double)yPrueba[i],
(double)aprox, (double)err);
    }
    if (res) std::fclose(res);

    // Curva en una malla fina de 2001 puntos para graficar.
    FILE* curva = std::fopen("curva_lagrange.csv", "w");
    if (curva) {
        std::fprintf(curva, "anio,poblacion\n");
        for (int i = 0; i <= 2000; i++) {
            real t = 29.0L * i / 2000.0L;
            std::fprintf(curva, "%.4f,%.2f\n", (double)(t + ANIO_INICIAL),
(double)lagrange(t, xNodos, yNodos));
        }
        std::fclose(curva);
    }

    // Tiempo de ejecución. El polinomio de Lagrange no tiene etapa de
    // construcción: todo el trabajo se hace en cada evaluación.
    const int REPS_PRUEBA = 100000, REPS_MALLA = 200;
    double tPrueba = medirMicrosegundos([&] {
        for (int anio : anioPrueba) sumidero = lagrange(anio - ANIO_INICIAL + cero, xNodos,
yNodos);
    }, REPS_PRUEBA);
    double tMalla = medirMicrosegundos([&] {
        for (int i = 0; i <= 2000; i++) sumidero = lagrange(29.0L * i / 2000.0L + cero,
xNodos, yNodos);
    }, REPS_MALLA);

    std::printf("\nTiempo de ejecucion (promedio por repeticion)\n");
    std::printf("%-34s %10s %16s\n", "Etapa", "Repet.", "Tiempo (us)");
    std::printf("%-34s %10d %16.3f\n", "Evaluacion en 5 anios de prueba", REPS_PRUEBA,
tPrueba);
    std::printf("%-34s %10d %16.3f\n", "Evaluacion en malla de 2001 puntos", REPS_MALLA,
tMalla);

    FILE* tiempos = std::fopen("tiempos_lagrange.csv", "w");
    if (tiempos) {
        std::fprintf(tiempos, "etapa,repeticiones,microsegundos\n");
        std::fprintf(tiempos, "prueba,%d,%.6f\n", REPS_PRUEBA, tPrueba);
        std::fprintf(tiempos, "malla,%d,%.6f\n", REPS_MALLA, tMalla);
        std::fclose(tiempos);
    }
    std::printf("\nArchivos generados: resultados_lagrange.csv, curva_lagrange.csv,
tiempos_lagrange.csv\n");
    return 0;
}
```

trazador_cubico.cpp

```
/*
 * trazador_cubico.cpp
 * -------------------
 * Interpolación de la población de Croacia (1994-2023) con un trazador
```

```
 * cúbico natural construido a partir de los 25 datos NO subrayados.
 *
 * En cada subintervalo [x_j, x_{j+1}]:
 *     S_j(x) = a_j + b_j (x - x_j) + c_j (x - x_j)^2 + d_j (x - x_j)^3,
 * con condiciones de frontera naturales S''(x_0) = S''(x_n) = 0.
 *
 * Los coeficientes c_j se obtienen de un sistema tridiagonal resuelto con
 * la factorización de Crout (algoritmo de Thomas), según el Algoritmo 3.4
 * de Burden & Faires, "Análisis Numérico".
 *
 * Compilar:  g++ -std=c++17 -O2 -o trazador_cubico trazador_cubico.cpp
 * Ejecutar:  ./trazador_cubico        (en Windows: trazador_cubico.exe)
 *
 * Salidas: tabla en consola, resultados_trazador.csv, curva_trazador.csv y
 * tiempos_trazador.csv (tiempo de ejecución del algoritmo).
 */

#include <chrono>
#include <cmath>
#include <cstdio>
#include <vector>

// ---------------------------------------------------------------------------
// Datos: población de Croacia 1994-2023 (hoja CROACIA del archivo Excel).
// ---------------------------------------------------------------------------
const int N_DATOS = 30;
const int ANIO_INICIAL = 1994;
const long POBLACION[N_DATOS] = {
    4675517, 4619362, 4562290, 4504627, 4445669, 4385823, 4326105, 4301079,
    4305783, 4306683, 4310050, 4313946, 4315802, 4315959, 4314635, 4310537,
    4301150, 4280827, 4259233, 4242403, 4219485, 4183732, 4136438, 4079198,
    4024868, 3986338, 3953956, 3924607, 3907031, 3896023};
// Años subrayados (resaltados en el Excel): se reservan para medir el error.
const int SUBRAYADOS[5] = {1995, 2000, 2008, 2015, 2022};

bool esSubrayado(int anio) {
    for (int s : SUBRAYADOS)
        if (s == anio) return true;
    return false;
}

// Separa los 30 datos en nodos de interpolación (25) y datos de prueba (5).
void separarDatos(std::vector<double>& xNodos, std::vector<double>& yNodos,
                  std::vector<int>& anioPrueba, std::vector<double>& yPrueba) {
    for (int i = 0; i < N_DATOS; i++) {
        int anio = ANIO_INICIAL + i;
        if (esSubrayado(anio)) {
            anioPrueba.push_back(anio);
            yPrueba.push_back(POBLACION[i]);
        } else {
            xNodos.push_back(anio);
            yNodos.push_back(POBLACION[i]);
        }
    }
}

// ---------------------------------------------------------------------------
// Algoritmo del trazador cúbico natural
// ---------------------------------------------------------------------------

// Coeficientes de los n polinomios cúbicos S_0, ..., S_{n-1}.
struct Trazador {
```

```
    std::vector<double> x, a, b, c, d;
};

Trazador trazadorNatural(const std::vector<double>& x, const std::vector<double>& y) {
    int n = (int)x.size() - 1;  // número de subintervalos
    std::vector<double> a = y;  // a_j = f(x_j)

    // Paso 1: longitudes de los subintervalos.
    std::vector<double> h(n);
    for (int i = 0; i < n; i++) h[i] = x[i + 1] - x[i];

    // Paso 2: lado derecho del sistema tridiagonal.
    std::vector<double> alfa(n + 1, 0.0);
    for (int i = 1; i < n; i++)
        alfa[i] = 3.0 / h[i] * (a[i + 1] - a[i]) - 3.0 / h[i - 1] * (a[i] - a[i - 1]);

    // Pasos 3-5: factorización de Crout y sustitución hacia adelante.
    // Las filas 0 y n codifican c_0 = c_n = 0 (frontera natural).
    std::vector<double> l(n + 1, 0.0), mu(n + 1, 0.0), z(n + 1, 0.0);
    l[0] = 1.0;
    for (int i = 1; i < n; i++) {
        l[i] = 2.0 * (x[i + 1] - x[i - 1]) - h[i - 1] * mu[i - 1];
        mu[i] = h[i] / l[i];
        z[i] = (alfa[i] - h[i - 1] * z[i - 1]) / l[i];
    }
    l[n] = 1.0;
    z[n] = 0.0;

    // Paso 6: sustitución hacia atrás para c, y cálculo de b y d.
    std::vector<double> c(n + 1, 0.0), b(n), d(n);
    for (int j = n - 1; j >= 0; j--) {
        c[j] = z[j] - mu[j] * c[j + 1];
        b[j] = (a[j + 1] - a[j]) / h[j] - h[j] * (c[j + 1] + 2.0 * c[j]) / 3.0;
        d[j] = (c[j + 1] - c[j]) / (3.0 * h[j]);
    }

    Trazador S;
    S.x = x;
    S.a.assign(a.begin(), a.end() - 1);
    S.b = b;
    S.c.assign(c.begin(), c.end() - 1);
    S.d = d;
    return S;
}

// Evalúa el trazador en t: localiza el subintervalo [x_j, x_{j+1}] y aplica S_j.
double evaluarTrazador(double t, const Trazador& S) {
    int n = (int)S.x.size() - 1;
    int j = n - 1;  // por defecto el último tramo (incluye t = x_n)
    for (int i = 0; i < n; i++) {
        if (S.x[i] <= t && t < S.x[i + 1]) {
            j = i;
            break;
        }
    }
    if (t < S.x[0]) j = 0;
    double dx = t - S.x[j];
    return S.a[j] + S.b[j] * dx + S.c[j] * dx * dx + S.d[j] * dx * dx * dx;
}

double errorRelativo(double exacto, double aprox) { return std::fabs(exacto - aprox) /
std::fabs(exacto); }
```

```cpp
// ---------------------------------------------------------------------------
// Medición del tiempo de ejecución
// ---------------------------------------------------------------------------

// Las variables volatile impiden que el compilador elimine o saque del ciclo
// de repeticiones los cálculos cuyo resultado no se usa.
volatile double sumidero;
volatile double cero = 0.0;

// Tiempo promedio, en microsegundos, de una ejecución de f. Se repite 'reps'
// veces porque una sola ejecución dura menos que la resolución del reloj.
template <typename F>
double medirMicrosegundos(F f, int reps) {
    auto inicio = std::chrono::steady_clock::now();
    for (int r = 0; r < reps; r++) f();
    auto fin = std::chrono::steady_clock::now();
    return std::chrono::duration<double, std::micro>(fin - inicio).count() / reps;
}

int main() {
    std::vector<double> xNodos, yNodos, yPrueba;
    std::vector<int> anioPrueba;
    separarDatos(xNodos, yNodos, anioPrueba, yPrueba);

    Trazador S = trazadorNatural(xNodos, yNodos);

    std::printf("Trazador cubico natural con %zu nodos - Croacia\n\n", xNodos.size());
    std::printf("%6s %14s %16s %16s\n", "Anio", "Exacto", "Trazador", "Error relativo");

    FILE* res = std::fopen("resultados_trazador.csv", "w");
    if (res) std::fprintf(res, "anio,exacto,aproximado,error_relativo\n");

    for (size_t i = 0; i < anioPrueba.size(); i++) {
        double aprox = evaluarTrazador(anioPrueba[i], S);
        double err = errorRelativo(yPrueba[i], aprox);
        std::printf("%6d %14.0f %16.2f %16.6e\n", anioPrueba[i], yPrueba[i], aprox, err);
        if (res) std::fprintf(res, "%d,%.0f,%.2f,%.6e\n", anioPrueba[i], yPrueba[i], aprox,
err);
    }
    if (res) std::fclose(res);

    // Curva en una malla fina de 2001 puntos para graficar.
    FILE* curva = std::fopen("curva_trazador.csv", "w");
    if (curva) {
        std::fprintf(curva, "anio,poblacion\n");
        for (int i = 0; i <= 2000; i++) {
            double t = ANIO_INICIAL + 29.0 * i / 2000.0;
            std::fprintf(curva, "%.4f,%.2f\n", t, evaluarTrazador(t, S));
        }
        std::fclose(curva);
    }

    // Tiempo de ejecución: construcción (cálculo de coeficientes) y evaluación.
    const int REPS_CONSTRUCCION = 100000, REPS_PRUEBA = 100000, REPS_MALLA = 2000;
    double tConstruccion = medirMicrosegundos([&] {
        Trazador T = trazadorNatural(xNodos, yNodos);
        sumidero = T.c[1] + cero;
    }, REPS_CONSTRUCCION);
    double tPrueba = medirMicrosegundos([&] {
        for (int anio : anioPrueba) sumidero = evaluarTrazador(anio + cero, S);
    }, REPS_PRUEBA);
```

```
    double tMalla = medirMicrosegundos([&] {
        for (int i = 0; i <= 2000; i++) sumidero = evaluarTrazador(ANIO_INICIAL + 29.0 * i /
2000.0 + cero, S);
    }, REPS_MALLA);

    std::printf("\nTiempo de ejecucion (promedio por repeticion)\n");
    std::printf("%-34s %10s %16s\n", "Etapa", "Repet.", "Tiempo (us)");
    std::printf("%-34s %10d %16.3f\n", "Construccion (coeficientes)", REPS_CONSTRUCCION,
tConstruccion);
    std::printf("%-34s %10d %16.3f\n", "Evaluacion en 5 anios de prueba", REPS_PRUEBA,
tPrueba);
    std::printf("%-34s %10d %16.3f\n", "Evaluacion en malla de 2001 puntos", REPS_MALLA,
tMalla);

    FILE* tiempos = std::fopen("tiempos_trazador.csv", "w");
    if (tiempos) {
        std::fprintf(tiempos, "etapa,repeticiones,microsegundos\n");
        std::fprintf(tiempos, "construccion,%d,%.6f\n", REPS_CONSTRUCCION, tConstruccion);
        std::fprintf(tiempos, "prueba,%d,%.6f\n", REPS_PRUEBA, tPrueba);
        std::fprintf(tiempos, "malla,%d,%.6f\n", REPS_MALLA, tMalla);
        std::fclose(tiempos);
    }
    std::printf("\nArchivos generados: resultados_trazador.csv, curva_trazador.csv,
tiempos_trazador.csv\n");
    return 0;
}
```

`graficas.py` (auxiliar, para las figuras)

```
"""
graficas.py
-----------
Genera las figuras del informe a partir de los archivos CSV que producen
los programas en C++ (lagrange.cpp y trazador_cubico.cpp).

Uso: primero ejecute ./lagrange y ./trazador_cubico; luego
     python graficas.py          (requiere matplotlib)
"""

import csv

import matplotlib

matplotlib.use("Agg")
matplotlib.rcParams["svg.fonttype"] = "path"
import matplotlib.pyplot as plt

AZUL, NARANJA, TINTA, GRIS = "#2a78d6", "#eb6834", "#0b0b0b", "#52514e"

# Datos exactos de Croacia 1994-2023 y años subrayados.
ANIOS = list(range(1994, 2024))
POB = [4675517, 4619362, 4562290, 4504627, 4445669, 4385823, 4326105, 4301079,
       4305783, 4306683, 4310050, 4313946, 4315802, 4315959, 4314635, 4310537,
       4301150, 4280827, 4259233, 4242403, 4219485, 4183732, 4136438, 4079198,
       4024868, 3986338, 3953956, 3924607, 3907031, 3896023]
SUBRAYADOS = [1995, 2000, 2008, 2015, 2022]
x_n = [a for a in ANIOS if a not in SUBRAYADOS]
y_n = [p / 1e6 for a, p in zip(ANIOS, POB) if a not in SUBRAYADOS]
x_p = SUBRAYADOS
y_p = [p / 1e6 for a, p in zip(ANIOS, POB) if a in SUBRAYADOS]
```

```
def leer(nombre):
    with open(nombre, newline="", encoding="utf-8") as f:
        return list(csv.DictReader(f))


curva_L = leer("curva_lagrange.csv")
curva_S = leer("curva_trazador.csv")
res_L = leer("resultados_lagrange.csv")
res_S = leer("resultados_trazador.csv")

plt.rcParams.update({
    "font.size": 10, "axes.spines.top": False, "axes.spines.right": False,
    "axes.edgecolor": GRIS, "axes.labelcolor": TINTA, "xtick.color": GRIS,
    "ytick.color": GRIS, "axes.grid": True, "grid.color": "#e4e3df",
    "grid.linewidth": 0.6, "legend.frameon": False,
})


def dibujar_base(ax):
    ax.plot([float(r["anio"]) for r in curva_L], [float(r["poblacion"]) / 1e6 for r in
curva_L],
            color=AZUL, lw=1.6, label="Polinomio de Lagrange (grado 24)")
    ax.plot([float(r["anio"]) for r in curva_S], [float(r["poblacion"]) / 1e6 for r in
curva_S],
            color=NARANJA, lw=1.6, ls="--", label="Trazador cúbico natural")
    ax.plot(x_n, y_n, "o", color=TINTA, ms=4, label="Datos usados (25 nodos)")
    ax.plot(x_p, y_p, "D", mfc="white", mec=TINTA, mew=1.4, ms=7,
            label="Datos subrayados (valor exacto)")
    ax.set_xlabel("Año")
    ax.set_ylabel("Población (millones)")


# Figura 1: vista completa.
fig, ax = plt.subplots(figsize=(7.5, 4.2))
dibujar_base(ax)
ax.set_title("Vista completa del intervalo 1994–2023", loc="left", fontsize=11)
ax.legend(loc="upper center", fontsize=8.5)
fig.tight_layout()
fig.savefig("fig_completa.svg")

# Figura 2: acercamiento a la escala de los datos.
fig, ax = plt.subplots(figsize=(7.5, 4.2))
dibujar_base(ax)
ax.plot(ANIOS, [p / 1e6 for p in POB], color=GRIS, lw=0.8, alpha=0.6, zorder=0)
ax.set_ylim(3.8, 4.75)
ax.set_xlim(1993.5, 2023.5)
ax.set_title("Acercamiento a la escala de los datos", loc="left", fontsize=11)
ax.legend(loc="upper right", fontsize=8.5)
fig.tight_layout()
fig.savefig("fig_acercamiento.svg")

# Figura 3: errores relativos (escala logarítmica).
eL = [float(r["error_relativo"]) for r in res_L]
eS = [float(r["error_relativo"]) for r in res_S]
fig, ax = plt.subplots(figsize=(7.5, 3.6))
pos = range(len(x_p))
ax.bar([p - 0.2 for p in pos], eL, width=0.38, color=AZUL, label="Lagrange")
ax.bar([p + 0.2 for p in pos], eS, width=0.38, color=NARANJA, hatch="//",
       edgecolor="white", label="Trazador cúbico")
ax.set_yscale("log")
ax.set_xticks(list(pos), [str(a) for a in x_p])
ax.set_xlabel("Año subrayado")
```

```
ax.set_ylabel("Error relativo (escala log)")
ax.legend(loc="upper center", ncol=2, fontsize=9)
ax.grid(axis="x", visible=False)
fig.tight_layout()
fig.savefig("fig_errores.svg")

# Tabla combinada para el informe.
with open("resultados.csv", "w", encoding="utf-8") as f:
    f.write("anio,exacto,lagrange,err_lagrange,trazador,err_trazador\n")
    for l, s in zip(res_L, res_S):
        f.write(f"{l['anio']},{l['exacto']},{float(l['aproximado']):.0f},"
                f"{float(l['error_relativo']):.3e},{float(s['aproximado']):.0f},"
                f"{float(s['error_relativo']):.3e}\n")

print("Generados: fig_completa.svg, fig_acercamiento.svg, fig_errores.svg, resultados.csv")
```